

Task-2: Spatial Vision and Localization

Pull the Task 2 in your existing local directory from [GitHub](#) .

1. Task 2A: Camera Calibration and ArUco Pose Estimation

Aim

The goal of this task is to use Python and OpenCV to:

- calibrate a camera lens to remove distortion,
- utilize ArUco marker dictionaries,
- and estimate the 3D pose (distance and offset) of physical markers from a 2D image.

This task simulates the localization pipeline used by autonomous rovers in advanced robotics scenarios.

Problem Statement

Cyborg is developing an autonomous rover. The rover navigates the outdoor course by scanning for physical waypoints marked with ArUco tags.

However, to maximize its field of view, the rover uses a wide-angle lens that introduces significant distortion. Before the rover can accurately judge the distance to a waypoint, it must mathematically correct this lens distortion using a pre-recorded calibration dataset.

Your task is to develop the camera calibration and localization node so that the robot can:

1. Read a set of checkerboard calibration images.
2. Calculate the camera matrix and distortion coefficients.
3. Undistort the live camera feed.
4. Detect all ArUco markers in the corrected image and calculate their exact distance from the camera.

Task Requirements

Your program must:

1. Read the 15 checkerboard images from the `calibration_images` folder (checkerboard inner grid: 9x6, square size: 2.5 cm).
2. Compute the camera matrix and return the trace (the sum of its main diagonal elements) to verify accuracy.
3. Load and undistort `test_arena.jpg` using the calculated coefficients.

4. Detect all ArUco markers using `DICT_4X4_50` on the undistorted image.
5. Use `solvePnP` to estimate the 3D pose of each marker, assuming the physical marker size is 5.0 cm.

Required Output Dictionary Format (Task 2A)

The final output returned by your function `localize_bot()` must strictly follow the dictionary format shown below.

- Keys must remain exactly the same.
- The `camera_matrix_trace` should be a float rounded to 2 decimal places.
- The marker dictionary keys must follow the format `id_<number>`.
- Distance and offset values should be in centimeters, rounded to 1 decimal place.

Example Output Format:

```
{
  "camera_matrix_trace": 1543.21,
  "markers": {
    "id_4": {"distance_z": 45.2, "x_offset": -12.4},
    "id_1": {"distance_z": 89.1, "x_offset": 5.2}
  }
}
```

Your evaluation depends on the exact structure of this dictionary.

Note: Following image undistortion, ArUco markers may appear visually warped or distorted. This is a normal consequence of the calibration process and should not be a cause for concern, as the transformation is intended to correct camera lens distortions rather than preserve the marker's original appearance.

The main task is to get familiar with “`cv2.findChessboardCorners(...)`” and use it over the “`test_arena.jpg`”. Even if the final image “`test_arena.jpg`” comes a bit distorted that should not be a matter of concern. It must detect the two ArUco ids only.

2. Task 2B: The Bird's-Eye Navigator and Real-World Mapping

Aim

The goal of this task is to use Python and OpenCV to:

- perform HSV color masking,
- calculate image moments and centroids,
- apply Perspective Transformation (Homography),

- and map pixel coordinates to real-world physical coordinates.

Problem Statement

In a separate project, Cyborg is testing a centralized tracking system for a multi-robot rescue scenario. An overhead camera is mounted on a wall overlooking a flat rectangular rescue arena. Because the camera is mounted at an angle, the video feed suffers from perspective distortion (the far side of the arena looks smaller than the near side).

The four corners of the arena floor are marked with colored tracking tape:

- **Top-Left:** Red
- **Top-Right:** Green
- **Bottom-Right:** Blue
- **Bottom-Left:** Yellow

A rescue bot with ArUco id 1 is driving on the arena floor.

Your task is to build a vision script that dynamically finds the four colored corners, stretches and warps the angled video feed into a perfectly flat 500x500 pixel "bird's-eye view" square, and calculates the robot's real-world physical coordinates in centimeters. **The centers of the coloured corners will be the corners for the required image.**

Task Requirements

Your program must:

1. Read the `angled_arena.png` image.
2. Use HSV masking and contour detection to find the center (centroid) of the four colored corners.
3. Store these coordinates in the exact order: [Top-Left, Top-Right, Bottom-Right, Bottom-Left].
4. Calculate a perspective transform matrix to warp the arena boundaries into a perfect 500x500 pixel image.
5. On this newly warped image, detect ArUco Marker ID 1 (`DICT_4X4_50`) and find its center pixel coordinate.
6. Assuming the 500x500 pixel warped image represents a physical arena of 200cm x 200cm (where the top-left corner is `[0.0, 0.0]`), scale the robot's pixel coordinates into physical centimeters.

Required Output Dictionary Format (Task 2B)

The final output returned by your function `map_arena()` must strictly follow the dictionary format shown below.

Example Output Format:

```
{
    "corner_points_detected": [[120, 50], [480, 70], [550, 400], [40,
380]],
    "robot_pixel_coord": [250, 300],
    "robot_real_world_coord": [100.0, 120.0]
}
```

Implementation and Evaluation

Write your logic in the `script2a.py` and `script2b.py` files at the designated places given in the template code.

To test your code, run `python3 main.py` inside the respective task folders. Check if the terminal prints out your expected dictionaries.

In task2B an image named “birdsvie.png” will be generated on running `main.py`. Save the image generated after successful run of your code as that will be used for submission.

IMPORTANT:

- You are only required to correctly populate the result dictionaries.
- Do NOT modify the function names, the dictionary structures, or the return formats.
- Only write your detection logic inside the designated sections of the scripts.

Evaluator

Once you have tested your scripts, run the evaluator binary to test your implementation on the hidden test cases. Execute the following command in the terminal from inside the `Task2a` or `Task2b` directory:

```
Bash
./evaluator
```

NOTE: Do not tamper with the evaluator file, as it may throw errors later on.

The evaluator will automatically test your implementation and display the final score. If your implementation passes the required test cases, the evaluator will ask for your roll number and generate an encrypted `.encrypt` submission file with your roll number as the file name.

DEADLINE: 29 May 2026